Spatial Logic Puzzles: Tetris – Exploring Strategies for the Highest Scoring Agents

Annie Wang Evan Wang Vivian Song Jonathan Calver Alice Gao

Department of Computer Science, University of Toronto

Overview

Objective: This study evaluates the performance of various Python-based AI agents and optimization algorithms within the spatial logic environment of Tetris.

- Feature weights are tuned to evaluate how different algorithms affect agent performance
- Explored various lookahead and pruning strategies for one agent

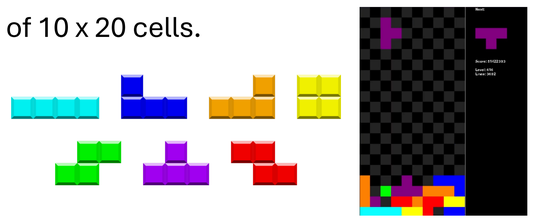
Rules

Environment: Game space is defined as a grid of 10 x 20 cells.

Tetrominos: There are seven distinct shapes

Piece Generation: Pieces are generated from a random uniform distribution

Scoring: Awarded more for getting a Tetris



Algorithms

One-step Lookahead with State-Space Search

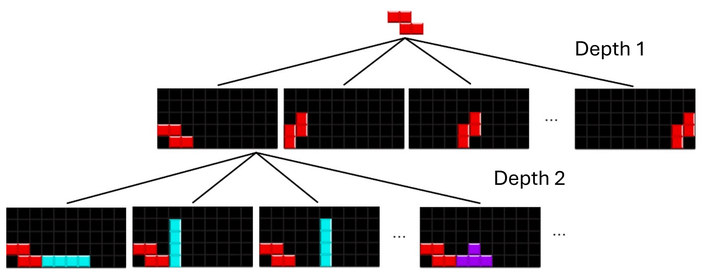
Calculates a score S for every possible piece placement of the current piece on the board: $S = w_1 f_1 + \ldots + w_n f_n$, where f_i is the i-th feature extracted from the board and w_i is the weight assigned to that feature

Feature Set:

- Aggregate Height
- Bumpiness
- Holes
- Line Clear Utility
- Column Transitions
- Row Transitions
- Maximum Height
- Cumulative Wells

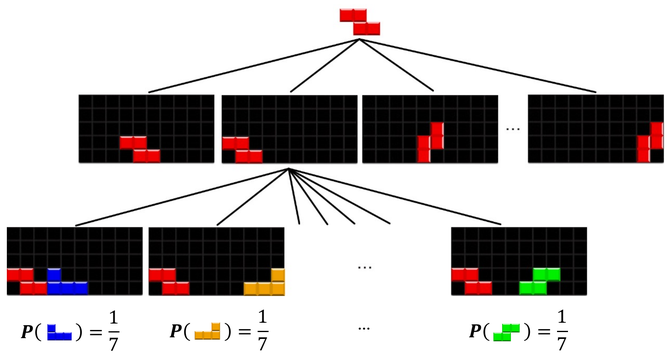
Depth Limited Search (DLS)

For a depth of d in decision tree, DLS simulates a sequence of d future pieces. Path with **highest score** is selected.



Depth Limited Expectimax

Calculates an expected score for every possible placement of current piece by simulating all future pieces

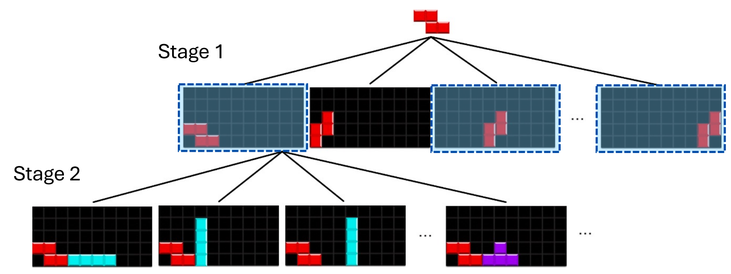


Two Stage Filtering (TSF)

Search optimization technique that prunes the state-space.

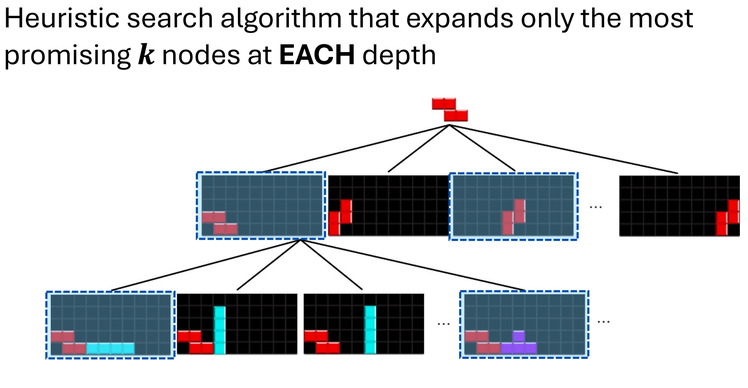
Stage 1: Choose k highest scoring paths using a **simplified** heuristic set

Stage 2: Only for chosen nodes, perform DLS with **full** heuristic set for remaining depth



Beam Search

Heuristic search algorithm that expands only the most promising k nodes at **EACH** depth



Optimizers

Cross Entropy Method (CEM)

CEM treats weights as a probability distribution and evolves them over multiple generations

1. **Sampling:** Generate a population of N candidates by sampling from a Gaussian distribution
2. **Evaluation:** Each candidate plays a set of games. The final score is recorded as its "fitness"
3. **Selection:** Identify top $X\%$ of candidates ("Elite" set)
4. **Updating:** Shift distribution's mean and variance toward Elite set

Genetic Algorithm (GA)

GA maintains a population of discrete agents, using Tournament Selection and Uniform Crossover to breed high-performing strategies.

1. **Selection:** Agents that achieve higher game scores are more likely to be chosen as "parents"
2. **Crossover:** Pairs of parents swap segments of their weight values to produce "offspring," combining successful traits from both
3. **Mutation:** Random noise is injected into weights to maintain genetic diversity and prevent population from getting stuck in local optima

Artificial Bee Colony (ABC)

Simulates a bee colony where bees explore flower patches (candidate solutions) with optimal fitness. The algorithm is split in 3 phases:

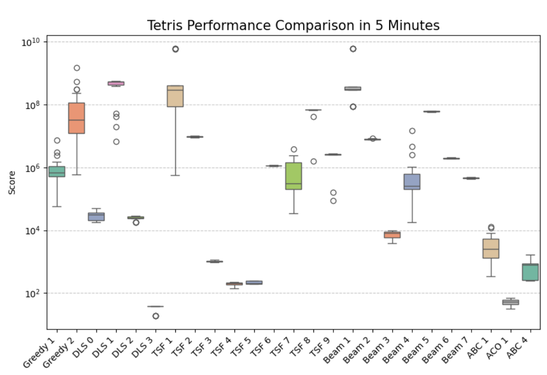
1. **Employed bee phase** (tweaks assigned source, keep if improve fitness)
2. **Onlooker bee phase** (explores promising solutions)
3. **Scout bee phase** (replace abandoned solution with random ones)

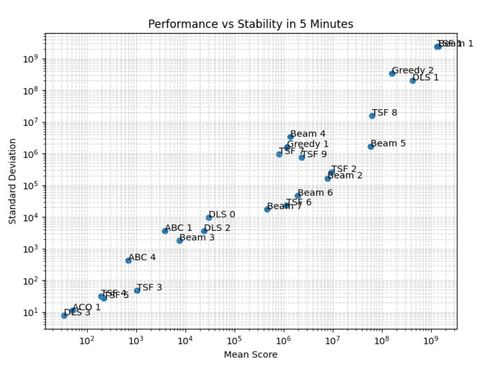
Ant Colony Optimization (ACO)

ACO builds candidate solutions, rewards strongest, and reinforces best choices through the following steps:

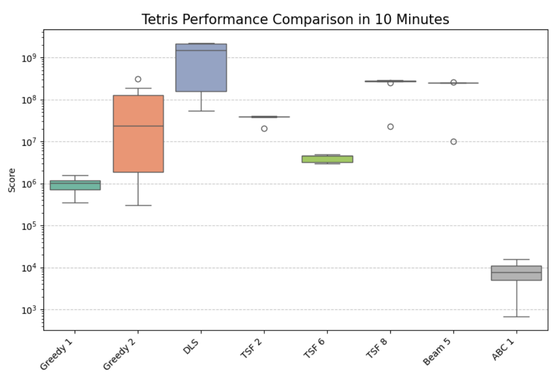
1. **Ant construction** (sample a candidate from pheromone)
2. **Evaluation** (score the candidate)
3. **Pheromone update** (reinforce good choices, evaporate old trails)

Analysis & Results

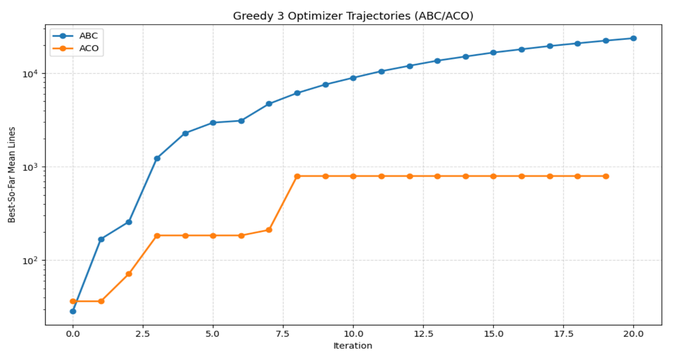




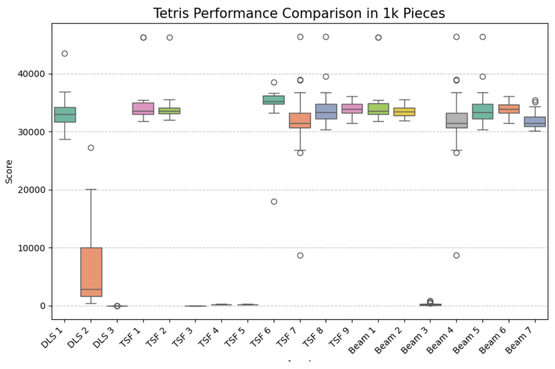
- TSF and Beam strategies achieve **dominance** compared with baseline agents
- TSF is a highly stable strategy

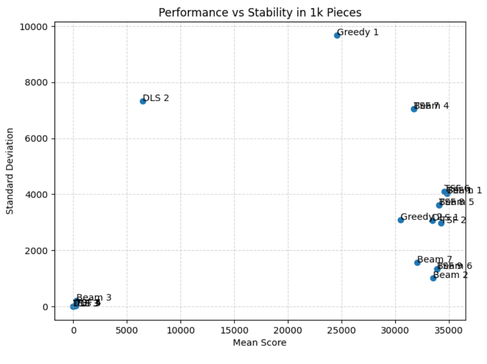


- Variants of lookahead strategies show high **"RNG-proof" consistency**

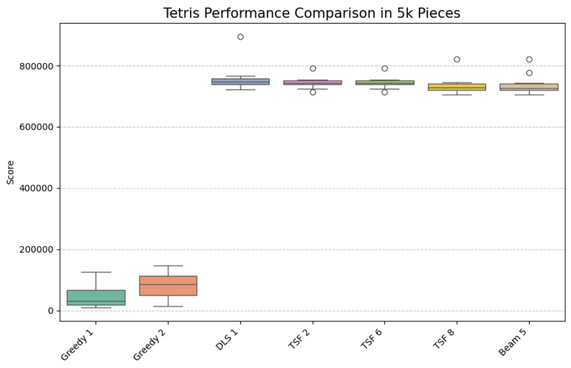


- ABC improves steadily and **reaches higher best-so-far mean lines** than ACO. ACO rises early but **plateaus quickly**

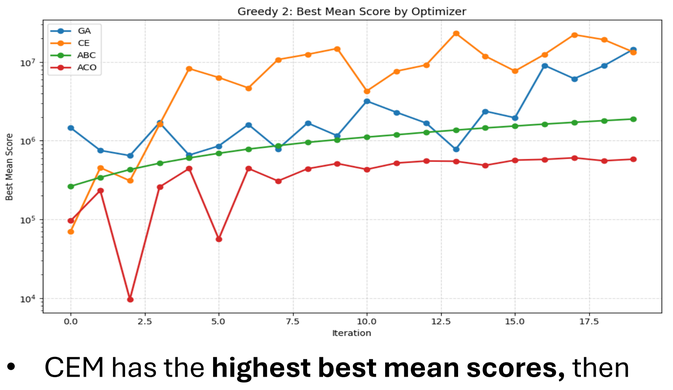




- Most agents exhibit **high precision and stability**
- Some agents fail to complete computation before pieces reach the board



- Lookahead is more critical than heuristic tuning for **long-term survival**



- CEM has the **highest best mean scores,** then GA with larger fluctuations. and ACO with the **weakest**

Algorithm	Pros	Cons
One-step Lookahead	• Extremely fast • Easy to tune weights	• Short-sighted • Often creates wells because of no lookahead
DLS	• Strategic placement • Higher ceiling	• Computation cost • Diminishing returns
TSF	• Ignores obviously bad paths, increasing efficiency	• Pruning risk (might discard good move) • Harder to balance threshold of which moves pass
Expectimax	• Probabilistic robustness	• Computationally heavy
Beam	• Controlled complexity • Deep reach without exponential explosion	• No optimality guarantee

References & Related Work

1. Christophe Thiery, Bruno Scherrer. Building Controllers for Tetris. International Computer Games Association Journal, 2009, 32, pp.3-11. ffinria-00418954f
2. Dunham, M., & Alves, A. (n.d.). Tetris game-playing agents in python. https://www.cs.uml.edu/ecg/uploads/AIfall14/dunham_alves_tetris_game_playing.pdf
3. Gilliland, T., & Zhang, D. (n.d.). Parallel greedy tetris solver. https://www.cs.columbia.edu/~sedwards/classes/2020/4995-fall/reports/Tetris.pdf
4. Jaivigneshvenugopal. (n.d.). *Jaivigneshvenugopal/AI-Tetris-Agent: A computer program that plays the game Tetris without any human aid.* GitHub. https://github.com/jaivigneshvenugopal/ai-tetris-agent
5. Lee, Y. (2017, June 14). *Tetris AI – the (near) perfect bot*. Code My Road. https://codemyroad.wordpress.com/2013/04/14/tetris-ai-the-near-perfect-player/
6. Naing, T., Truong, A., & Zeng, O. (n.d.). Tetris AI. https://thawsitt.me/files/Tetris_AI_CS221_final_paper.pdf